Week-5: Spark Fundamentals -- Data Cleaning, Transformation & Aggregation

Dataset used: customer_transactions.csv — 20,500 e-commerce transaction records with duplicates, null values, empty strings, and inconsistent date formats. All queries below were executed on this dataset using PySpark.

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

MapReduce processes data in two rigid stages (Map and Reduce) and writes intermediate results to disk after every stage. This repeated disk I/O makes it slow for iterative algorithms such as machine learning and graph analytics. Spark overcomes these limitations by processing data in memory whenever possible, using a Directed Acyclic Graph (DAG) execution engine that optimizes the entire workflow before execution. Spark also provides unified libraries for SQL, Streaming, Machine Learning, and Graph Processing, reducing development effort while improving performance and scalability.
Spark is preferred because it minimizes disk I/O and optimizes execution using DAG scheduling.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Spark caches frequently used datasets in RAM using cache() or persist(). Iterative algorithms repeatedly access the same data; keeping it in memory avoids repeated disk reads and writes, reducing latency and significantly improving performance.
Iterative machine learning algorithms such as gradient descent, k-means, and PageRank reuse the same dataset across many iterations.

Disk-based systems such as MapReduce read input from HDFS at every iteration, process it, and write the output back to disk. With n iterations, this results in n full read and write cycles, causing massive I/O overhead.

```
from pyspark.sql import functions as F

df = spark.read.csv("customer_transactions.csv", header=True,
inferSchema=True)
df.cache()

for i in range(10):
    model = train_step(df)
```

Spark's DAG scheduler also pipelines transformations and only recomputes lost partitions using lineage if a node fails, giving fault tolerance without disk persistence overhead.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

dropDuplicates() : it removes duplicate records based on selected columns while preserving one occurrence.

```
df_clean = df.dropDuplicates(["user_id", "transaction_date"])
```

Actual result on customer_transactions.csv:

```
Row count before dedup : 20500
Row count after dedup  : 17988
Duplicate rows removed  : 2512

+--------------+-------+----------------+
|transaction_id|user_id|transaction_date|
+--------------+-------+----------------+
|T0001700      |U000002|2025-10-06      |
|T0003702      |U000002|31/07/2024      |
|T9000740      |U000003|2026-01-03      |
|T0017773      |U000004|03-03-2026      |
|T0016161      |U000004|12-29-2025      |
+--------------+-------+----------------+
```

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

```
result = (
    df_sales
    .filter(df_sales.region == "West")
    .groupBy("product_category")
    .agg(F.avg("sale_amount").alias("avg_sale_amount"))
)
result.show()
```

Actual result on customer_transactions.csv:

```
+----------------+------------------+
|product_category|avg_sale_amount   |
+----------------+------------------+
|Beauty          |383.02            |
|Home & Kitchen  |381.48            |
|Sports          |369.32            |
|Electronics     |365.78            |
|Books           |341.52            |
|Groceries       |341.12            |
|Clothing        |339.03            |
+----------------+------------------+
```

Q5: What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string 'Unknown'.

.na.drop(): it removes rows containing null values, dropping the entire row.

.na.fill(): it replaces null values with a specified default, keeping the row.

.na.drop(): it reduces row count and can lose valid data in other columns. .na.fill() preserves rows while imputing missing values.

```
df_filled = df.na.fill({"status": "Unknown"})
```

Actual result on customer_transactions.csv:

```
Null status values before fill : 1025
Rows with status = 'Unknown' after fill : 1025
```

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

```
result = (
    df.groupBy("city")
    .count()
    .filter(F.col("count") > 100)
)
result.show()
```

Filtering happens after groupBy and agg using .filter(), which is equivalent to SQL's HAVING clause, not .where() applied before grouping.

Actual result on customer_transactions.csv:

```
+----------+-----++-----------+-----+
|city      |count|
+-----------+-----+
|Delhi     |1383 |
|Bangalore |1333 |
|Kochi     |1298 |
|Kolkata   |1296 |
|Mumbai    |1274 |
|Chennai   |1273 |
|Surat     |1270 |
|Hyderabad |1247 |
|Guwahati  |1245 |
|Jaipur    |1245 |
|Lucknow   |1223 |
|Ahmedabad |1222 |
```

```
|Pune       |1222 |
|Patna      |1200 |
|Chandigarh |1196 |
|Bhubaneswar|1163 |
```

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Spark DataFrames are immutable. Every transformation returns a new DataFrame, leaving the original unchanged. This design enables lineage-based fault tolerance, lazy evaluation, and safe parallel execution.

```
df2 = df.drop("first_name")
df3 = df2.withColumnRenamed("sale_amount", "total_sale")
```

Actual result on customer_transactions.csv:

```
Original df still has 'first_name' column? True
df2 (after drop) has 'first_name' column?  False
df3 columns (after rename): [..., 'price', 'total_sale', 'status',
...]
```

Transformations must be chained or reassigned, such as df = df.filter(...).drop(...), otherwise the changes are lost.

This immutability enables lineage tracking for fault tolerance and safe parallel and distributed execution, since there is no risk of concurrent writes corrupting shared state.

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
result = df.filter((F.col("age").between(18, 30)) &
(F.col("subscription") == "Premium"))
```

Actual result on customer_transactions.csv:

```
Matching rows: 1459

+--------------+----+------------+
|transaction_id|age |subscription|
+--------------+----+------------+
|T0017745      |22.0|Premium     |
|T0012287      |21.0|Premium     |
|T0017114      |28.0|Premium     |
|T0002133      |23.0|Premium     |
|T0016793      |20.0|Premium     |
```

```
+--------------+----+------------+
```

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Functions such as sum() and avg() ignore nulls by default in Spark SQL, which can silently skew results if nulls represent something meaningful, such as an unknown value rather than a true zero.

Nulls can distort denominators, since avg() divides by the count of non-null values rather than the total row count, so results can look correct while actually misrepresenting the dataset.

Nulls can also propagate through calculated columns, such as price multiplied by quantity producing null if either value is null, which corrupts downstream aggregates.

Actual result on customer_transactions.csv (price column has 1230 nulls):

```
avg(price) ignoring nulls (default Spark behavior) : 104.66
avg(price) after na.fill({'price': 0})              : 98.38

The two answers differ by roughly 6, showing how silently ignoring
nulls
vs explicitly treating them as 0 changes the reported average.
```

Cleaning nulls first, either by dropping or filling with a sensible default such as 0, ensures the aggregation reflects an explicit and intentional decision rather than Spark's implicit null-skipping behavior, making results reproducible and auditable.

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

```
from pyspark.sql.types import TimestampType

df = df.withColumn("raw_timestamp",
F.col("raw_timestamp").cast(TimestampType())) \
       .withColumnRenamed("raw_timestamp", "event_time")
```

Because raw_timestamp in this dataset contains multiple formats, a direct cast() leaves unmatched formats as null. In practice this needs a multi-format parse:

```
df = df.withColumn(
    "raw_timestamp",
    F.coalesce(
        F.to_timestamp(F.col("raw_timestamp"), "yyyy-MM-dd
HH:mm:ss"),
        F.to_timestamp(F.col("raw_timestamp"), "yyyy-MM-
dd'T'HH:mm:ss"),
```

```
        F.to_timestamp(F.col("raw_timestamp"), "dd/MM/yyyy HH:mm")
    )
).withColumnRenamed("raw_timestamp", "event_time")
```

Actual result on customer_transactions.csv:

```
event_time column dtype: timestamp

+--------------+-------------------+
|transaction_id|event_time         |
+--------------+-------------------+
|T0016735      |2025-01-15 02:43:00|
|T0009434      |2024-10-16 15:21:40|
|T0016209      |2026-03-28 15:27:52|
|T9001399      |2026-01-29 01:24:53|
+--------------+-------------------+
```

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Shuffle is the process of redistributing data across partitions and nodes so that all records with the same key end up on the same partition. It is required for operations such as groupBy, join, distinct, and repartition.

Steps involved in a shuffle: data is read from partitions across the cluster; records are hashed by key and written to disk or network buffers; data is transferred across the network to the executor that owns each key's partition; the receiving executor reads and merges the shuffled data before applying the aggregation.

A wide transformation is one where each output partition depends on multiple input partitions, unlike a narrow transformation such as filter or map, where each output partition depends on only one input partition. Since groupBy needs to bring together records with the same key from potentially every partition, it requires a full shuffle, which is expensive in terms of network I/O, disk spill, and serialization, and is a common performance bottleneck in Spark jobs.

Actual physical plan for df.groupBy('region').count() (the Exchange step is the shuffle):

```
-- Physical Plan --
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[region], functions=[count(1)])
   +- Exchange hashpartitioning(region, 200), ENSURE_REQUIREMENTS
      +- HashAggregate(keys=[region], functions=[partial_count(1)])
         +- FileScan csv [region] ...

The 'Exchange hashpartitioning' line is the shuffle boundary;
records
```

```
are redistributed by hash of the region key before the final
aggregation.
```

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

```
df_clean = df.filter(
    F.col("email").isNotNull() &
    F.col("username").isNotNull() &
    (F.trim(F.col("username")) != "")
)
```

Actual result on customer_transactions.csv (email has 820 nulls; username has empty-string and whitespace-only entries):

```
Row count before cleaning : 20500
Row count after cleaning  : 18893
Rows removed              : 1607
```

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

```
result = df.agg(
    F.min("price").alias("min_price"),
    F.max("price").alias("max_price"),
    F.avg("price").alias("mean_price")
)
result.show()
```

Actual result on customer_transactions.csv:

```
+---------+---------+------------+
|min_price|max_price|mean_price  |
+---------+---------+------------+
|5.63     |639.25   |104.66      |
+---------+---------+------------+
```

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

inferSchema samples the data and guesses types automatically. If date formats are inconsistent, such as mixing MM/DD/YYYY, YYYY-MM-DD, and free text like N/A, Spark may fall back to inferring the column as StringType instead of DateType or TimestampType, silently disabling date-based operations.

It may also misparse some rows, producing null for unparseable formats without any warning, and cause schema mismatches across files if different files or batches have different date formats, leading to inconsistent types across a unioned dataset.

Actual result on customer_transactions.csv:

```
transaction_date column, with inferSchema=True, was inferred as:
string
(not DateType) because the column mixes multiple real formats:

+----------------+
|transaction_date|
+----------------+
|01/11/2024      |
|2024-08-20      |
|06-12-2025      |
|08-Sep-2024     |
+----------------+
```

The safer practice is to explicitly define the schema using StructType and use to_date() or to_timestamp() with explicit format strings, then validate and clean malformed entries deliberately rather than relying on silent inference.

Q15: Write a final processing pipeline that: 1) Filters out duplicates, 2) Fills null prices with 0, 3) Groups by store_id to calculate total revenue.

```
pipeline_result = (
    df_sales
    .dropDuplicates()
    .na.fill({"price": 0})
    .groupBy("store_id")
    .agg(F.sum("price").alias("total_revenue"))
    .orderBy(F.col("total_revenue").desc())
)

pipeline_result.show()
```

Actual result on customer_transactions.csv (60 stores total; top 10 shown):

```
+--------+-------------+
|store_id|total_revenue|
+--------+-------------+
|S006    |38895.86     |
|S037    |38013.21     |
|S023    |37302.66     |
|S030    |37086.64     |
|S022    |36916.70     |
|S028    |36868.56     |
```

```
|S039    |36839.79     |
|S019    |36041.69     |
|S051    |35987.82     |
|S041    |35831.46     |
+--------+-------------+
```

Key Insights

1. Spark is faster than MapReduce due to in-memory execution and DAG optimization.
2. DataFrames are immutable and lazily evaluated.
3. Clean nulls and duplicates before aggregation.
4. Shuffles are expensive; minimize unnecessary wide transformations.
5. Explicit schemas are preferred over inferSchema in production.